Algorithmic Robotics
COMP/ELEC/MECH 450/550
Project 3 Part 1: Random Tree Planner

Darshon Singh (ds161) and Winson Lin (wl85)

Problem

We implemented the Random Tree Planner (RTP) in C++ as an OMPL planner and used it to plan collision free motions for the seven joint Panda arm from Project 2. The planner only talks to OMPL's state space, validity checker and goal, so it doesn't know anything about the arm. We then made two of our own 3D scenes with sphere and box obstacles, picked start and goal configurations where the straight joint space line between them hits an obstacle, and ran RTP on them.

Panda arm and its joint space

The Panda is a 7 joint revolute arm. The base and gripper are fixed, so a configuration is just the seven joint angles

$$q = (q_1, \ldots, q_7), \qquad \mathcal{C} = \prod_{i=1}^{7} [q_i^{min}, q_i^{max}] \subset \mathbb{R}^7$$

with these limits from the URDF (radians):

joint	1	2	3	4	5	6	7
min	-2.97	-1.83	-2.97	-3.14	-2.97	-0.09	-2.97
max	2.97	1.83	2.97	0.09	2.97	3.82	2.97

This is a box in $\mathbb{R}^7$, set up as `RealVectorStateSpace(7)`. Local motions are straight lines between joint vectors. The arm geometry for collision checking is 59 spheres attached to the links. Forward kinematics puts them in the world frame and they get checked against the scene spheres and oriented boxes. Self collision isn't modeled. The longest diagonal of the joint box is about 13.4 rad, so the fixed edge resolution of 0.005 times that means states along an edge are checked about every 0.067 rad.

RTP implementation

The tree is a `std::vector` of `Motion` structs, each holding a state and a parent pointer. Since RTP picks tree vertices uniformly there is no need for a nearest neighbor structure, a random index into the vector is enough. Each iteration picks a random vertex q_a, samples q_b uniformly (or uses the goal with probability equal to the goal bias), and calls `checkMotion(`q_a`,` q_b`)` on the whole edge. If the edge is valid, q_b is cloned and added with q_a as its parent. The loop stops when a new vertex satisfies the goal or the termination condition fires. The path is recovered by walking parent pointers back to the start and reversing.

While planning we keep track of the vertex with the smallest goal distance. If time runs out, we return the path to that vertex as an approximate solution. The planner owns every state and `Motion` it allocates, and `freeMemory()` releases all of them. Both `clear()` and the destructor call it, which matters because the benchmark reuses one planner instance across trials. `getPlannerData()` adds every tree edge and labels the start vertex and the vertex that reached the goal.

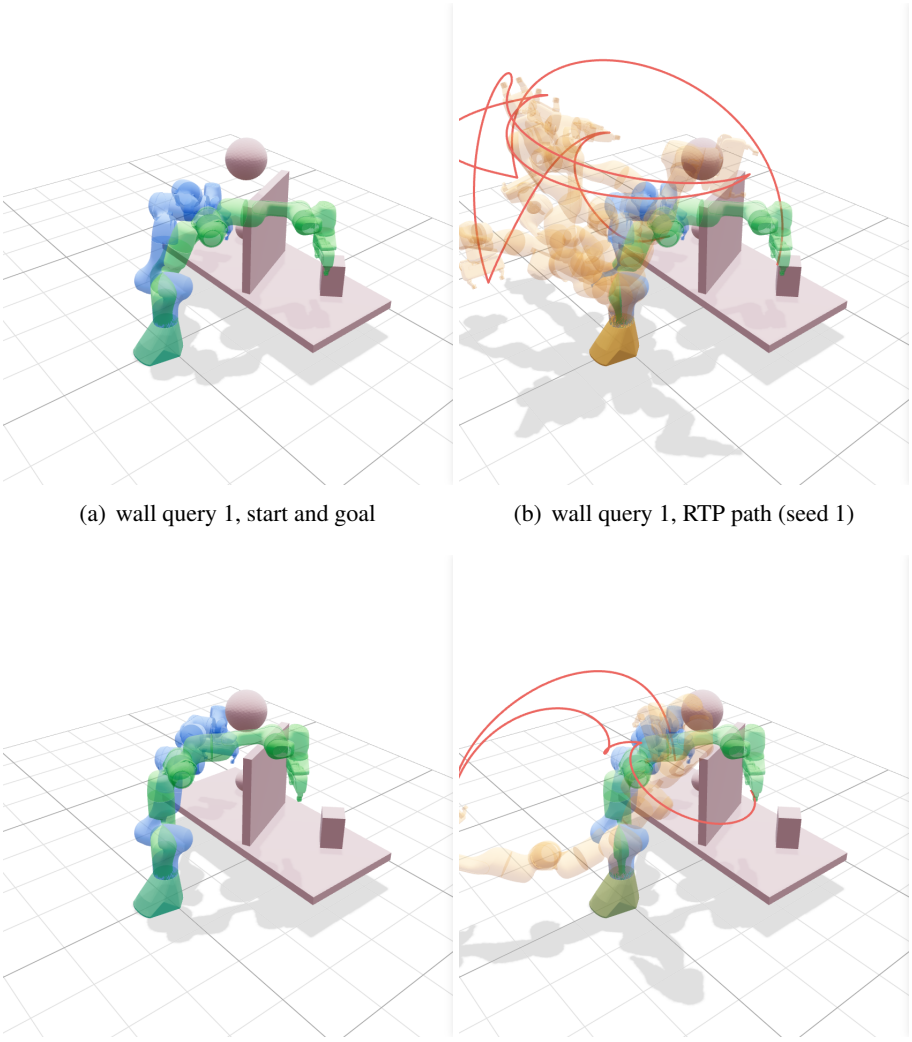
To check correctness we ran RTP on all five queries of the supplied clutter and shelf scenes with 5 seeds each, then densely rechecked every edge of each returned path at a step of 0.002 rad and checked that the planner data had $|V|-1$ edges. All 50 runs were exact and started and ended at the right configurations. One shelf path had a spot where the dense recheck found about 1 mm of penetration between two of the 0.067 rad validator steps. This comes from the fixed motion checking resolution and not from RTP itself, and we kept the resolution fixed like the handout says. With a 2 ms time limit RTP correctly returns an approximate solution.

Custom scenes

Both scene files are in `resources/scenes/`. In all figures the blue arm is the start, the green arm is the goal, the faded orange arms are the intermediate waypoints of the RTP path, and the red curve is the path of the gripper.

Wall scene (`wall.yaml`): a table in front of the robot with a thin wall standing on it, rotated about 25 degrees around z, which splits the table into a left ($+y$) side and a right ($-y$) side. There is a ball hanging just above the wall at a height of 0.78 m so the arm can't just go straight over the top of it, plus a small ball on the left side and a rotated block on the right side as clutter. Both queries start with the hand pointing down on the left side of the wall and end on the right side. In query 1 the hand goes from $(0.46, 0.37, 0.27)$ to $(0.62, -0.29, 0.39)$ m and in query 2 from $(0.54, 0.42, 0.39)$ to $(0.51, -0.23, 0.52)$ m. The straight joint space line swings the forearm through the wall, so the direct edge is invalid for both.

Window scene (`window.yaml`): a window frame in the plane $x = 0.45$ m with an opening about 0.6 m wide and 0.5 m tall. The right post is tilted 15 degrees about x so the opening isn't a perfect rectangle. Behind the window there is a small shelf with a ball on it. In query 1 the arm starts rotated off to the left of the robot (hand near $(0.07, 0.55, 0.37)$) and has to reach through the window to $(0.56, 0.08, 0.44)$. In query 2 it starts off to the right (hand near $(0.20, -0.45, 0.38)$) and reaches higher through the window to $(0.62, 0.08, 0.50)$. Both goals only have about 4 to 5 cm of clearance, and the direct edge hits the frame for both.



(a) wall query 1, start and goal

(b) wall query 1, RTP path (seed 1)

(c) wall query 2, start and goal

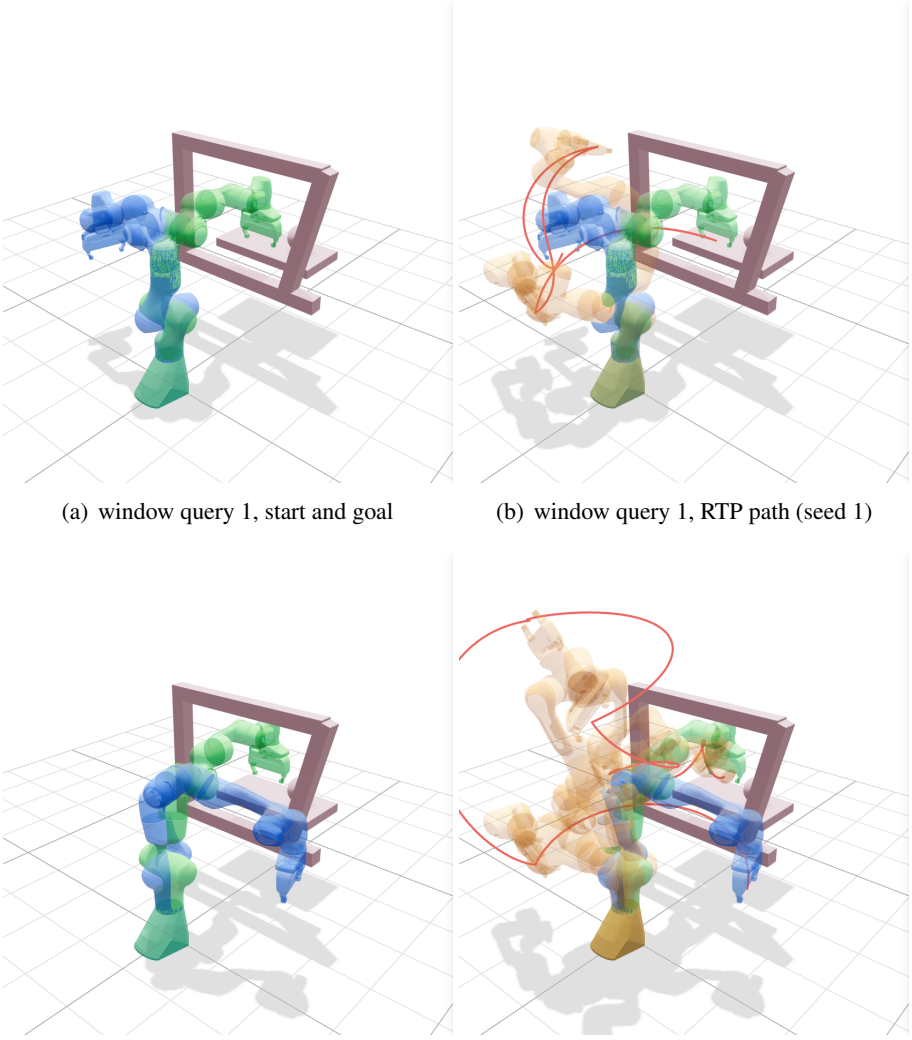
(d) wall query 2, RTP path (seed 5)

Figure 1: wall scene in viser

RTP paths and performance

The example paths in Figures 1 and 2 were made with `plan_arm.py` using goal bias 0.05 and a 10 s limit, and are saved in `results/`. To get a feel for how reliable RTP is we ran it with seeds 1 to 10 on each custom query and seeds 1 to 5 on each supplied query, all with goal bias 0.05 and 10 s. Time, length and waypoints are medians over the exact runs only.

scene	exact	time (s)	length (rad)	waypoints	tree size
clutter (supplied)	25/25	0.30	11.6	5	26
shelf (supplied)	25/25	0.36	19.2	6	31
wall	20/20	0.56	21.9	6	51
window	14/20	2.55	24.5	7	209



(a) window query 1, start and goal

(b) window query 1, RTP path (seed 1)

(c) window query 2, start and goal

(d) window query 2, RTP path (seed 1)

Figure 2: window scene in viser

The wall scene was easy for RTP even though the direct edge is blocked, since there is a lot of free space around and over the wall and only a few long edges are needed. The window scene was a lot harder. Query 1 was solved 9/10 times but query 2 only 5/10, and the failed runs had built trees of around 800 vertices before time ran out. To get through the window the whole forearm has to be lined up with the opening, and since RTP connects to a random vertex with a full length edge, almost all of those long edges clip the frame and get thrown away. The tree basically keeps adding vertices out in the open space in front of the robot and rarely gets one through the gap.

In a lot of the failed runs the approximate path was just the start configuration. We thought this was a bug at first, but in 7D the start is usually closer to the goal than any uniform sample in the joint box, so the closest tree vertex really is the start.

The paths themselves look pretty bad, which is what we expected from RTP. Every edge goes to a uniformly random configuration, so the arm makes big swings around the base and the gripper traces large loops (the red

curves) before ending up at the goal. For comparison the straight line distance between start and goal is only 1.2 to 1.6 rad for our queries, while the RTP paths were around 20 to 25 rad long, so about 15 times longer than needed. The saved wall query 1 path is 30.8 rad over 8 waypoints. Paths also pass close to obstacles since nothing pushes them away. Minimum clearance on the saved paths was 0.5 to 3.7 cm. Every path is valid, but none of them is a motion you would want to run on a real arm without shortcutting first.

The main implementation details we had to be careful about were memory ownership (clone the sampled state before adding it, free everything in `clear()`) and returning the right approximate solution. For running everything we used the course OMPL environment in a Docker container. One thing we noticed is that OMPL only lets you set the seed before the first random number is drawn, so repeated planning calls in one Python process aren't reproducible. Because of that we ran each seed as its own `plan_arm.py` or Python process.

Contributions

[CONTRIBUTIONS]